Evaluating Mitigation Robustness Under Multi-Turn Prompt Injection: Detection Latency, Context Drift, and Transferability

Jibran Sheikh (js08312) Syeda Wania (sh08450)

Research Question

How do mitigation strategies perform under increasing conversational pressure, and how early do they detect adversarial escalation across varying context lengths and attack types?

Abstract

Prompt-injection defenses for large language models are typically benchmarked using single-turn attacks and summarized with Attack Success Rate. This misses three properties that matter in deployed systems: when a defense activates during an evolving attack, how its reliability changes as context grows, and whether it generalizes beyond the attack patterns it was tuned against. We propose a systematic evaluation framework that measures **Detection Latency**, the turn delay before a mitigation flags or blocks adversarial escalation; **Context-Length Drift**, the rate at which safety performance degrades as conversation length increases; and **Attack–Defense Transferability**, the drop in mitigation effectiveness when evaluated on unseen attack families. We evaluate defenses across short, medium, and long conversations, stress-testing three mitigation classes against direct, multi-turn, and indirect injection attacks. We hypothesize that defenses tuned for single-turn benchmarks degrade under sustained conversational pressure, exhibiting delayed detection and poor generalization to unseen attack families. Our findings provide deployment-ready insights, helping practitioners choose mitigations that minimize exploitable detection gaps while avoiding excessive over-refusal of legitimate queries.

Motivation & Impact

As LLMs are deployed in long, multi-turn settings such as customer service and healthcare, delayed threat detection creates exploitable windows. Current benchmarks do not measure when defenses activate or how they degrade with context length. This work addresses that gap, providing practical insights for security teams, safety researchers, and policymakers to design timely, context-aware mitigation strategies.

Related Work & Gap

Prior work shows that LLM defenses weaken in multi-turn, long-context settings as accumulated context erodes safety enforcement (Pathade, 2025). The threat model has evolved from direct jailbreaks to indirect and agent-based attacks (Wang et al., 2025), with automated red-teaming methods such as RedHit (Sorkhpour et al., 2025) and SAP frameworks (Deng et al., 2023) enabling adaptive multi-round attacks. However, evaluations largely rely on Attack Success Rate and do not measure detection timing, context-driven degradation, or defense generalization. We address these gaps by introducing detection latency, context-length drift, and attack–defense transferability as structured evaluation metrics.

Proposed Method

We systematically evaluate LLM robustness against multi-turn prompt injection by testing three attack families (direct, multi-turn escalation, indirect) against three mitigation strategies (prompt hardening, input/output gating, and conversation-state monitoring). Experiments are conducted across short, medium, and long conversation lengths to assess context effects. Performance is measured using Attack Success Rate, detection latency, context-length drift, and defense transferability, while also tracking over-refusal to ensure security does not overly restrict legitimate use.

Experiment Plan

We evaluate defenses using 150 custom multi-turn conversations alongside public benchmarks (HarmBench, AdvBench), comparing no mitigation, single mitigations, and combined defenses. Success is defined by improved detection latency, low transferability gaps across attack families, and minimal context-driven safety drift. The evaluation is implemented through a Python-based harness using GPT-4 and Claude/Gemini APIs with turn-level logging for detailed analysis.

Deliverables

A public GitHub repo with all code and a research paper with results and visualizations.

Schedule & Milestones

By Week 7, attack families, mitigation strategies, and evaluation metrics will be finalized. In week 8, the evaluation harness will be implemented. Weeks 9-12 will integrate APIs, run baseline and mitigation experiments, and collect metrics such as ASR, detection latency, and context drift. The final phase will focus on analysis, visualization, paper writing, and releasing the GitHub repository and research paper.